AI Coding Goal Harness

0. 使用原则

每次只运行一个 Harness。不要把所有 Harness 一次性发给模型。

每轮至少显式引用：`@README.md @Plan.md @STATE.md`

需要改代码时，再显式引用相关源码和 sample testcase。

统一规则：

- README 是最高优先级。
- 不依赖历史聊天记忆。
- 不允许因为"代码写完"而结束。
- 只有测试、格式、核心需求均通过后才能进入 `SUBMIT_READY`。
- 遇到不确定：重新读 README → sample testcase → 现有代码。
- 遇到失败：定位 → 修改 → 重跑，禁止直接退出。

Harness 1：需求拆解

Goal

读取 README、sample testcase 和项目结构，生成可靠 `Plan.md`，不要开始大规模编码。

Prompt

```
你正在执行持续 Goal，不是回答单轮问题。

先阅读：
@README.md
@sample_testcases/...
@项目入口和主要源码

任务：
1. 找出程序入口、输入、输出、评分方式、必须实现项。
2. 阅读 sample testcase，确认真实输入输出和验证方式。
3. 检查现有项目结构，找出需要修改的文件。
4. 创建或更新 @Plan.md。

Plan.md 只保留：
- Goal
- 输入/输出协议
- 必须实现项（R1/R2/...）
- 边界条件
- 实现步骤
```

```
- 测试要求
- 未确认项

每个 Requirement 尽量可以判断 PASS/FAIL。
README 优先于你的常识。
不要进行大规模编码。
完成后创建/更新 @STATE.md，写清 NEXT ACTION。
```

Harness 2：Plan Reviewer

Goal

检查 Plan 是否遗漏 README/sample 的硬要求。

Prompt

```
你是严格 Reviewer。

先阅读：
@README.md
@Plan.md
@sample_testcases/...

逐条检查：
1. README 每个必须要求是否在 Plan.md 中有对应项。
2. sample testcase 是否暴露了 Plan 未覆盖的接口/格式/边界。
3. 是否存在错误假设、数据泄漏风险、输入输出遗漏。
4. 是否遗漏程序入口、文件路径、列名、顺序、类型要求。

不要泛泛提优化建议。
发现问题直接修改 @Plan.md。
不要重新设计整个系统。

完成后更新 @STATE.md：
CURRENT PHASE=IMPLEMENT
NEXT ACTION=最优先实现项
```

Harness 3：实现任务

Goal

一次只完成一个明确需求，并立即验证。

Prompt

```
你正在执行持续 Goal。

先阅读：
@README.md
@Plan.md
@STATE.md
@本轮相关源码
@相关 sample testcase

根据 STATE.md 的 NEXT ACTION 工作。

规则：
1. 先确认该任务对应 README/Plan 哪个 Requirement。
2. 阅读现有实现后再修改，禁止猜代码。
3. 优先最小修改，避免无关重构。
4. 修改后立即运行最相关测试。
5. 如果失败：读真实错误→定位→修改→重跑。
6. 不得因为一种方案失败而退出。
7. 完成后更新 Plan.md 对应 PASS/FAIL 状态。
8. 更新 STATE.md：COMPLETED、CURRENT FAILURES、FILES CHANGED、TEST STATUS、NEXT ACTION。

除非全部 DONE CONDITIONS 满足，否则禁止宣布完成。
```

Harness 4：测试与修复

Goal

把“看起来能跑”变成“真实能跑”。

Prompt

```
先阅读：
@README.md
@Plan.md
@STATE.md
@sample_testcases/...
@被测试源码

执行测试 Gate：

A. 官方 sample testcase
B. 输入输出协议检查
C. README 相关边界条件
D. 从正式入口执行 clean run

重点检查：
- 路径
- 列名/顺序
- row 数
```

```
- dtype
- NaN/inf
- 未见类别
- 时间排序
- 随机性
- 临时文件/缓存依赖
- debug 输出污染结果

任何 Gate 失败:
立即修复并重跑，不要只写分析报告。

更新 @STATE.md 的 TEST STATUS 和 CURRENT FAILURES。
只要仍有 FAIL/UNKNOWN，就不能进入 SUBMIT_READY。
```

Harness 5：最终隐藏测试审计

Goal

模拟隐藏测试作者，找最后的失败点。

Prompt

```
先阅读:
@README.md
@Plan.md
@STATE.md
@sample_testcases/...
@主要实现文件

假设你是隐藏测试作者。

逐项检查:
1. README Requirement 是否全部真实实现。
2. 是否有硬编码 sample 的行为。
3. 输入 schema 改变是否会崩。
4. 缺失值、极值、单行、未知类别是否安全。
5. 时间特征是否存在未来泄漏。
6. test 输出是否保持原始顺序。
7. clean environment 是否可直接运行。
8. 是否依赖 notebook 状态、旧缓存或人工步骤。

发现问题：直接修改代码并运行测试。

只有以下全部通过时才允许:
CURRENT PHASE=SUBMIT_READY

条件:
- README 必须项全 PASS
- sample 全通过
```

```
- I/O 协议通过
- clean run 通过
- 无已知 runtime error
- 无必须项 FAIL/UNKNOWN
```

Harness 6：提交反馈修复

Goal

有限提交次数下，只根据真实反馈做高置信度修复。

Prompt

```
先阅读：
@README.md
@Plan.md
@STATE.md
@平台提交反馈
@相关源码

不要重写整个方案。

先把失败归类：
- runtime
- format
- correctness
- robustness
- metric/performance

然后：
1. 找最可能对应的 Requirement。
2. 尽量本地复现。
3. 做最小修改。
4. 重跑 sample。
5. 重跑 clean run。
6. 和 known-good 版本比较。
7. 只有高置信度改善后才再次提交。

禁止因为可能提升分数而破坏已通过的接口和稳定性。
更新 STATE.md 的 SUBMISSION 和 NEXT ACTION。
```

STATE.md 最小模板

```
# CURRENT PHASE
PLAN / IMPLEMENT / VERIFY / REPAIR / SUBMIT_READY
```

```
# CURRENT GOAL
...

# COMPLETED
- ...

# CURRENT FAILURES
- ...

# FILES CHANGED
- ...

# TEST STATUS
sample:
format:
edge:
clean_run:

# NEXT ACTION
...

# KNOWN-GOOD VERSION
...

# SUBMISSION
remaining_or_unknown:
last_result:
```

SCADA 回归题附加 Harness

仅在题目属于历史 SCADA + 气象 → 未来功率预测时使用。

Prompt

```
在正常 README/Plan 检查之外，额外确认：
- target 列
- timestamp
- train/valid/test schema
- 1h horizon 定义
- 机组 ID
- lag 是否会未来泄漏
- 天气字段对应当前还是未来
- 缺失值
- 类别编码
- valid 指标
- test 输出顺序

优先顺序：
1. 先跑通可靠 baseline。
2. sample / I-O / clean run 全通过。
```

```
3. 再看 valid metric。
4. 再增加合法时间特征、历史特征、统计特征。
5. 最后才调 LightGBM/XGBoost 参数。

禁止一开始就做 Optuna、stacking、深度模型。
README 若有不同规定，以 README 为准。
```

推荐轮次分配

- 第1轮：Harness 1
- 第2轮：Harness 2
- 第3-8轮：Harness 3
- 第9-10轮：Harness 4
- 第11轮：Harness 5
- 剩余轮次：Harness 6，用于提交反馈修复

上下文不足时，不要补长解释；重新显式 `@README.md @Plan.md @STATE.md @相关源码` 即可。